

Chapter 8

Attention & Transformers in Vision

The introduction of attention mechanisms marked a paradigm shift in deep learning, moving away from rigid sequential recurrence (RNNs) and localized spatial receptive fields (CNNs) toward dynamic, content-dependent context aggregation. Originally designed to eliminate information bottlenecks in neural machine translation, the Query-Key-Value (QKV) attention framework enabled models to dynamically weigh and attend to arbitrary input tokens in parallel.

When adapted to computer vision through the Vision Transformer (ViT), this architecture challenged long-held assumptions by demonstrating that convolutional inductive biases—such as translation invariance and spatial locality—are not strictly required for vision tasks when given sufficient data or effective distillation. This chapter explores the theoretical foundation of scaled dot-product and multi-head attention, details the mechanics of ViTs and patch embeddings, and examines key architectural advancements like DeiT (Data-efficient Image Transformers) and Swin Transformers (hierarchical shifted-window attention) that bridge the gap between global context and computational efficiency.

8.1 Introduction

The idea of attention mechanisms traces back to an elegant observation: traditional sequence-to-sequence models, particularly encoder-decoder RNNs used for machine translation, suffered from a critical limitation. These models were forced to **compress** entire input sequences—potentially containing thousands of tokens into a single fixed-length vector. This bottleneck was fundamentally problematic. As sequences grew longer, the network struggled to retain information from early positions, a phenomenon now well-understood through the lens of vanishing gradients.

The core innovation of attention offered an elegant solution to this constraint. Rather than forcing the model to compress all information into a fixed vector, attention allows the decoder to dynamically **“look back” at different parts of the input** sequence at each generation step. This selective focus is precisely what attention mechanisms enable.

Attention mechanisms embody a simple yet powerful principle: not all information is equally relevant at every moment. Rather than compressing entire sequences into fixed-length vectors (as traditional encoder-decoder RNNs did), attention allows models to **dynamically assign importance weights to different input parts**. This selective weighting is differentiable, flexible, and enables parallel computation, which are key advantages over sequential RNN processing.

8.1.1 The Query-Key-Value Framework

Understanding attention begins with a database analogy. In a lookup operation:

- A **query vector** q represents one information request.
- Each **key vector** k_i helps locate one entry.
- Each **value vector** v_i contains the information associated with k_i .

Let $\mathcal{D} = \{(k_i, v_i)\}_{i=1}^m$ denote a **database** of m key-value pairs. The query $q \in \mathbb{R}^{d_k}$ is a vector with d_k components, not a scalar. Each key $k_i \in \mathbb{R}^{d_k}$ is also a vector, while each value $v_i \in \mathbb{R}^{d_v}$ may have a different dimension. The lowercase symbol v_i denotes one value vector, while the capital matrix

$$K = \begin{pmatrix} k_1^\top \\ \vdots \\ k_m^\top \end{pmatrix} \in \mathbb{R}^{m \times d_k}, \quad V = \begin{pmatrix} v_1^\top \\ \vdots \\ v_m^\top \end{pmatrix} \in \mathbb{R}^{m \times d_v}$$

contains all keys and values. Attention compares q with every row of K and computes a weighted combination of the rows of V :

$$e_i = q^\top k_i, \quad \alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^m \exp(e_j)}, \quad \text{attention}(q, K, V) = \sum_{i=1}^m \alpha_i v_i.$$

The dot product $q^\top k_i$ produces one scalar score e_i , and softmax converts these scores into scalar attention weights α_i . The weights sum to one, and their weighted combination of the vectors v_i produces one output vector in $\mathbb{R}^{d_v}$.

8.1.2 Scaled Dot-Product Attention

In practice, attention mechanisms often use **scaled dot-product attention**. Given a query vector q , key vectors k_i , and value vectors v_i , the scalar scores and attention weights are:

$$e_i = \frac{q^\top k_i}{\sqrt{d_k}}, \quad \alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^m \exp(e_j)}, \quad \text{attention}(q, K, V) = \sum_{i=1}^m \alpha_i v_i,$$

where d_k is the dimensionality of the queries and keys. The scaling factor $1/\sqrt{d_k}$ controls the magnitude of the scores and helps prevent softmax from entering regions with very small gradients.

Note

When there are n_q queries, they are stacked as the rows of $Q \in \mathbb{R}^{n_q \times d_k}$. With $K \in \mathbb{R}^{n_k \times d_k}$ and $V \in \mathbb{R}^{n_k \times d_v}$, all queries are processed at once:

$$\text{attention}(Q, K, V) = \text{softmax}_{\text{row}} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V,$$

producing an output in $\mathbb{R}^{n_q \times d_v}$. Thus, q is one query and Q contains all queries; similarly, v_i is one value and V contains all values.

For a batch of B independent sequences, a leading batch dimension is added:

$$Q \in \mathbb{R}^{B \times n_q \times d_k}, \quad K \in \mathbb{R}^{B \times n_k \times d_k}, \quad V \in \mathbb{R}^{B \times n_k \times d_v}.$$

The same formula is applied independently to each batch element using batched matrix multiplication; attention does not mix different sequences in the batch. Here, $V[b, i, :]$ is the i -th value vector of the b -th sequence. In self-attention, Q , K , and V are projections of the same input sequence and $n_q = n_k$.

Example: Scaled Dot-Product Attention: Full Matrix Calculation

Given the 4-token sequence sentence = [“The”, “cat”, “drinks”, “milk”], here is the full mathematical formulation of scaled dot-product attention.

This example contains one sequence, so there is no batch dimension. The four tokens produce $n_q = n_k = 4$ query, key, and value vectors.

1. Query (Q) and Key (K) Matrices

Each token is projected into a query vector $q_i \in \mathbb{R}^{d_k}$ and a key vector $k_i \in \mathbb{R}^{d_k}$. Stacking these vectors as rows forms $Q, K \in \mathbb{R}^{4 \times d_k}$. Using $d_k = 2$:

$$Q = \begin{pmatrix} q_{\text{The}}^T \\ q_{\text{cat}}^T \\ q_{\text{drinks}}^T \\ q_{\text{milk}}^T \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 3 & 2 \\ 2 & 4 \\ 4 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} k_{\text{The}}^T \\ k_{\text{cat}}^T \\ k_{\text{drinks}}^T \\ k_{\text{milk}}^T \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 3 \\ 1 & 2 \\ 3 & 0 \end{pmatrix}.$$

Each entry of $QK^\top \in \mathbb{R}^{4 \times 4}$ is a scalar dot product between one query and one key.

2. Value Matrix (V) and Attention Weights (A)

Each token also has a value vector $v_i \in \mathbb{R}^{d_v}$. These are learned projections used to construct the output. Stacking all four value vectors as rows gives $V \in \mathbb{R}^{4 \times d_v}$. Using $d_v = 2$:

$$V = \begin{pmatrix} 1 & 0 \\ 2 & 4 \\ 0 & 2 \\ 5 & 1 \end{pmatrix}$$

First, the scaled score matrix is:

$$S = \frac{QK^\top}{\sqrt{d_k}} \approx \begin{pmatrix} 0.7071 & 3.5355 & 2.1213 & 2.1213 \\ 2.1213 & 8.4853 & 4.9497 & 6.3640 \\ 1.4142 & 11.3137 & 7.0711 & 4.2426 \\ 2.8284 & 7.7782 & 4.2426 & 8.4853 \end{pmatrix}$$

Applying softmax independently to each row produces $A \in \mathbb{R}^{4 \times 4}$:

$$A = \text{softmax}_{\text{row}}(S) \approx \begin{pmatrix} 0.03825 & 0.64711 & 0.15732 & 0.15732 \\ 0.00150 & 0.86901 & 0.02533 & 0.10417 \\ 0.00005 & 0.98496 & 0.01415 & 0.00084 \\ 0.00231 & 0.32633 & 0.00951 & 0.66184 \end{pmatrix}.$$

Each row corresponds to one query vector, and its four scalar weights sum to approximately one.

3. Computation of the Final Output Matrix

Multiplying the attention matrix A by the value matrix V gives one output vector for each query:

$$\text{attention}(Q, K, V) = AV \approx \begin{pmatrix} 2.1191 & 3.0604 \\ 2.2604 & 3.6308 \\ 1.9742 & 3.9690 \\ 3.9642 & 1.9862 \end{pmatrix}.$$

4. Meaning of the Final Matrix

The resulting 4×2 matrix contains the four output vectors. For example, the first row is the updated representation associated with the query for “The”:

$$0.03825v_{\text{The}} + 0.64711v_{\text{cat}} + 0.15732v_{\text{drinks}} + 0.15732v_{\text{milk}} \approx (2.1191 \quad 3.0604).$$

In this head, the first output vector assigns weight ~ 0.04 to the value vector associated with “The”, ~ 0.65 to the one of “cat”, ~ 0.16 to the one of “drinks” and ~ 0.16 to the one of “milk”. The same calculation is applied to the remaining three queries.

8.1.3 Multi-Head Attention

Rather than attending through a single representation, **multi-head attention** computes attention in parallel across h different learned subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O,$$

where

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V),$$

and W_i^Q, W_i^K, W_i^V, W^O are learned projection matrices. Here, d_{model} is the dimensionality of each token representation used by the model; in other words, an input sequence of n tokens has shape $n \times d_{\text{model}}$ before the head-specific projections. A common choice is $d_k = d_v = d_{\text{model}}/h$. The concatenated heads then have width $hd_v = d_{\text{model}}$, and W^O combines their information into the final representation.

Ideally, different heads focus on different aspects of the input, allowing the model to represent several attention patterns simultaneously. However, this specialization is not guaranteed: some heads may learn similar or redundant patterns.

8.1.4 Positional Encoding

Since attention mechanisms are inherently permutation-equivariant, they require a way to incorporate the order of tokens in a sequence. This is achieved through **positional encodings**, which inject information about the position of each token into the model.

A common approach is to use **sinusoidal functions**:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right); \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right),$$

where pos is the position and i is the dimension. These encodings are added to the input embeddings, allowing the model to distinguish between different positions in the sequence.

Modern variants of transformers often use **learned positional embeddings** instead, which are trained alongside the model parameters. For vision transformers, learnable positional embeddings are typically preferred, as they can adapt to the specific characteristics of visual data.

8.1.5 The Transformer Architecture

The original Transformer removes recurrence and convolution, using attention together with feed-forward networks. It follows an encoder-decoder design (see Figure 8.1):

- **Encoder:** It consists of a stack of N identical layers, where each layer is composed of two sublayers:
 - A multi-head self-attention mechanism that allows the model to attend to different positions in the input sequence simultaneously.
 - A fully connected feed-forward network consisting of two linear transformations with Rectified Linear Unit (ReLU) activation in between:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2.$$

The output is an updated representation matrix of the sequence with the same dimension ($m \times d$) as the input.

Each of these two sublayers has a residual connection followed by layer normalization:

$$\text{LayerNorm}(x + \text{sublayer}(x)).$$

- **Decoder:** It also consists of N identical layers, where each layer is composed of three sublayers:
 - A masked multi-head self-attention mechanism that prevents positions from attending to subsequent (future) tokens.
 - A multi-head cross-attention mechanism in which queries come from the decoder, while keys and values come from the encoder output.
 - A fully connected feed-forward network that has the same function as in the encoder.

Residual connections and normalization are also applied around each decoder sublayer.

- **Output layer:** For sequence generation, a linear projection followed by softmax produces a probability distribution over the vocabulary at each decoder position.
- **Positional encoding:** Sinusoidal functions or learned position embeddings added directly to the input and target embeddings prior to the encoder and decoder stacks to inject sequence order information.

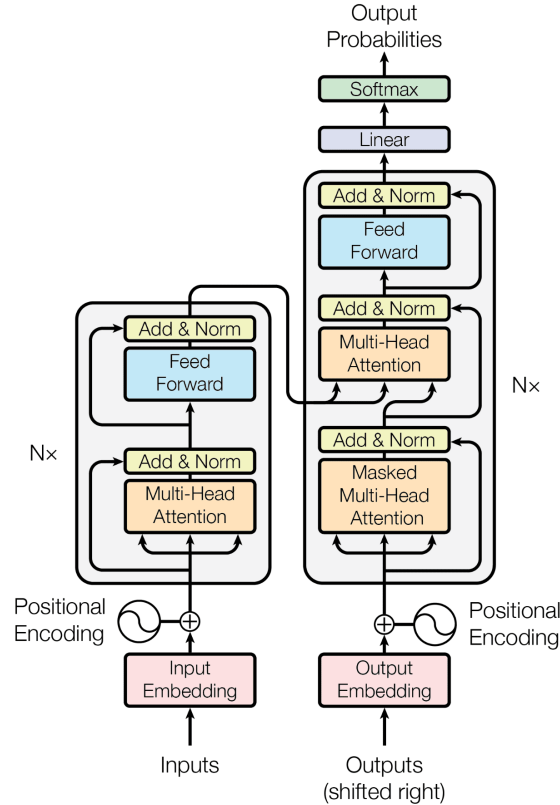


Figure 8.1: Transformer architecture.

8.2 Vision Transformer (ViT)

The remarkable success of Transformers in NLP raised a natural question: can we apply transformers to vision tasks? The naive approach—treating each pixel as a token—would result in prohibitively expensive quadratic complexity. The key insight, introduced in Vision Transformer (ViT), is to treat image patches as tokens, not pixels.

8.2.1 Architecture Overview

Vision Transformer (ViT) adapts the Transformer encoder to image classification through the following pipeline (also shown in Figure 8.2):

1. **Patch embedding:** Divide the image into fixed-size patches.
2. **Linear projection:** Project each patch to a fixed-dimensional embedding.
3. **Positional encoding:** Add learnable position information.
4. **Transformer encoder:** Process the patch sequence through n identical encoder layers.
5. **Classification head:** Use a special [CLS] token for classification.

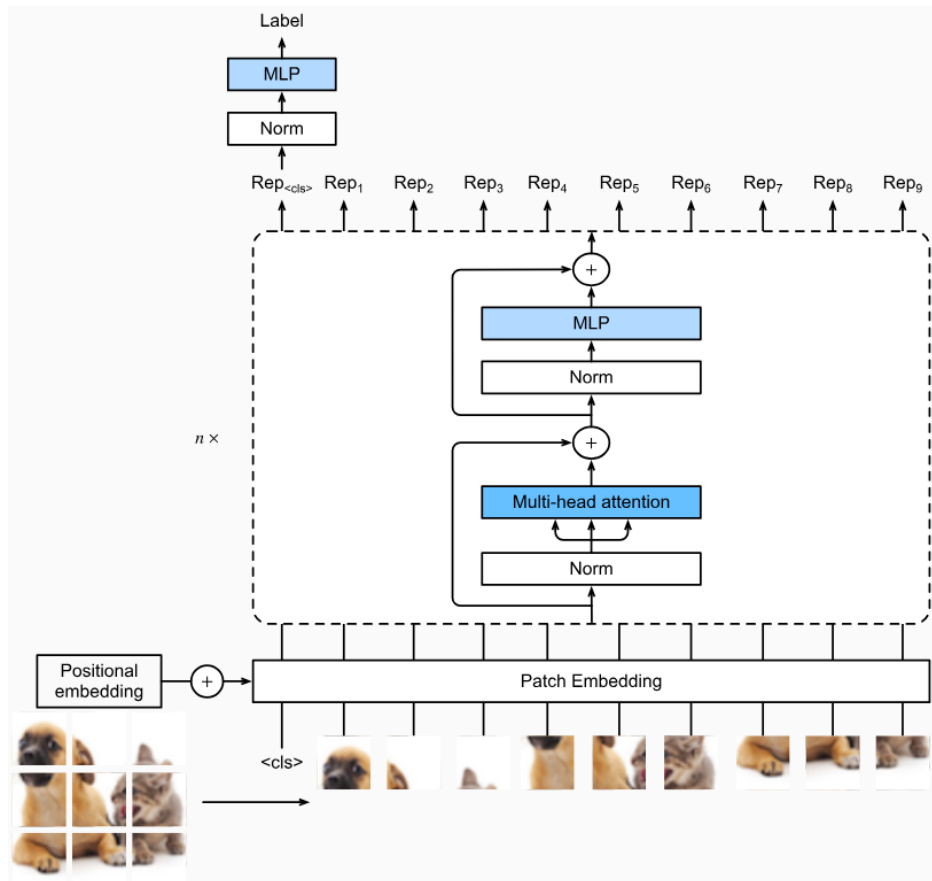


Figure 8.2: Vision transformer architecture.

Unlike the original post-normalized Transformer described above, ViT applies layer normalization before each attention and MLP sublayer and uses GELU in its MLP.

8.2.2 Patch Embedding: Tokenizing Images

A crucial design choice in ViT is how to convert images to token sequences. For an image of size $H \times W \times C$ (height $\times$ width $\times$ channels) with patch size P , assuming that P divides both H and W :

- The image is divided into $N = \frac{H}{P} \frac{W}{P} = \frac{HW}{P^2}$ non-overlapping patches of size $P \times P$.
- Each patch p_i is flattened into a vector in $\mathbb{R}^{P^2 C}$.
- A learned matrix $E \in \mathbb{R}^{(P^2 C) \times D}$ projects each patch to a token of dimension $D = d_{\text{model}}$.

This operation can be implemented with a single convolution having kernel size P , stride P , no padding, and D output channels. For a batch of B images, it produces a tensor of shape $(B, D, H/P, W/P)$, which is flattened and transposed to (B, N, D) .

Example: Patch Embedding Example

A 256×256 RGB image with 16×16 patches produces $N = (256 \cdot 256)/(16 \cdot 16) = 256$ patch tokens. Each patch initially contains $3 \cdot 16 \cdot 16 = 768$ raw pixel values, which are projected to dimension D .

Formally, the embedded representation $x_i \in \mathbb{R}^D$ of the flattened patch p_i is:

$$x_i = p_i E + b,$$

where $p_i \in \mathbb{R}^{768}$, $E \in \mathbb{R}^{768 \times D}$, and $b \in \mathbb{R}^D$.

In practice, a convolution with kernel size 16×16 , stride 16, no padding, and D output channels converts the image into a $16 \times 16 \times D$ feature map, which is reshaped into $X \in \mathbb{R}^{256 \times D}$.

Patching reduces the sequence from 65,536 pixel locations to 256 patch tokens, making attention much more practical. It does not necessarily reduce the total number of scalar values; for example, when $D = 768$, both representations contain 196,608 values.

8.2.3 The [CLS] Token

The **[CLS] token** is a learnable vector $x_{\text{cls}} \in \mathbb{R}^D$ that interacts with all patch tokens through self-attention. After passing through the Transformer encoder stack, its final representation is used as input to the classification head, typically a linear layer. The [CLS] token is a design choice; averaging the final patch representations is another possible approach.

This design is surprisingly effective: the attention mechanism naturally learns to aggregate relevant information from all patches into the [CLS] token's representation.

8.2.4 Positional Encoding

Transformers are **permutation-equivariant**: without position information, the model cannot distinguish between different spatial arrangements of patches. ViT adds a learnable positional embedding to every token, including [CLS]. The complete input sequence is:

$$z_0 = [x_{\text{cls}}; x_1; \dots; x_N] + E_{\text{pos}}, \quad E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}.$$

8.2.5 Classification

Now we have a sequence of $N + 1$ embeddings (one [CLS] token and N patch tokens), each of dimension D , with positional information. This sequence is passed through the Transformer encoder:

- At layer ℓ , self-attention receives all token representations from layer $\ell - 1$. The current [CLS] representation can attend to every patch and itself, while each patch can attend to all other tokens.
- The transformer encoder refines the representations across n layers (typically 12 or 24). After passing through all n encoder layers, we extract the representation of the [CLS] token.
- The final [CLS] representation, which has dimension D , is passed through a classification head. The linear layer produces one logit per class, and softmax converts these logits into class probabilities.

8.2.6 Training Challenges and Practical Considerations

Despite its elegance, ViT revealed important challenges that continue to influence modern architectures:

- **Data efficiency:** Unlike CNNs, which have strong inductive biases (locality, translation invariance), transformers have minimal built-in assumptions about visual structure. Consequently, ViTs trained on small datasets significantly underperform CNNs. ViTs require either large-scale pre-training (on datasets like ImageNet-21k with 14 million images or JFT-300M with 300 million images) or sophisticated data augmentation strategies.
- **Computational cost:** The quadratic complexity of attention in image resolution is problematic. For a 256×256 image with 16×16 patches, we have 256 embeddings, leading to $256 \times 256 = 65536$ attention operations. For high-resolution images, this becomes prohibitive.
- **Training stability:** Large Transformers can be challenging to train, requiring careful hyperparameter tuning and initialization strategies.

8.3 Extensions

Challenges commented on the previous section have motivated numerous extensions and improvements. We will look at the two most common ones.

8.3.1 Data-efficient Image Transformers (DeiT)

Data-efficient Image Transformers (DeiT) showed that a strong training and augmentation strategy can train a competitive ViT on ImageNet-1K without external data. DeiT also introduced an optional **teacher-student** distillation mechanism.

The distilled variant uses a learnable **distillation token** (x_{dist}) along with the standard class token (x_{cls}) in the input patch sequence:

$$X = [x_{\text{cls}}, x_1, \dots, x_N, x_{\text{dist}}]$$

During forward propagation, both x_{cls} and x_{dist} attend to all image patches via self-attention, and at the output layer:

- x_{cls} is supervised by the ground-truth label using standard Cross-Entropy loss.
- x_{dist} is supervised by either the hard prediction or the soft probability distribution of a pre-trained teacher model.

Let z_{cls} , z_{dist} , and z_t denote the logits from the student classification head, student distillation head, and teacher, respectively. The teacher is fixed during student training. DeiT can use either of the following losses.

For **hard distillation**, the teacher target is its predicted class:

$$\mathcal{L}_{\text{hard}} = \lambda \text{CE}(z_{\text{cls}}, y) + (1 - \lambda) \text{CE}(z_{\text{dist}}, \arg \max\{z_t\}).$$

For **soft distillation**, the student matches the complete teacher distribution:

$$p_t = \text{softmax}(z_t), \quad p_{\text{dist}} = \text{softmax}(z_{\text{dist}}),$$

$$\mathcal{L}_{\text{soft}} = \lambda \text{CE}(z_{\text{cls}}, y) + (1 - \lambda) \text{KL}(p_t \parallel p_{\text{dist}}),$$

Hard distillation is simple and directly transfers the teacher's preferred class; it also performed best in the original DeiT experiments. Soft distillation transfers more information by preserving the teacher's confidence and relationships among classes.

Here, y is the ground-truth label and $\lambda \in [0, 1]$ controls the balance between the two terms. At inference, the predictions of the classification and distillation heads can be averaged.

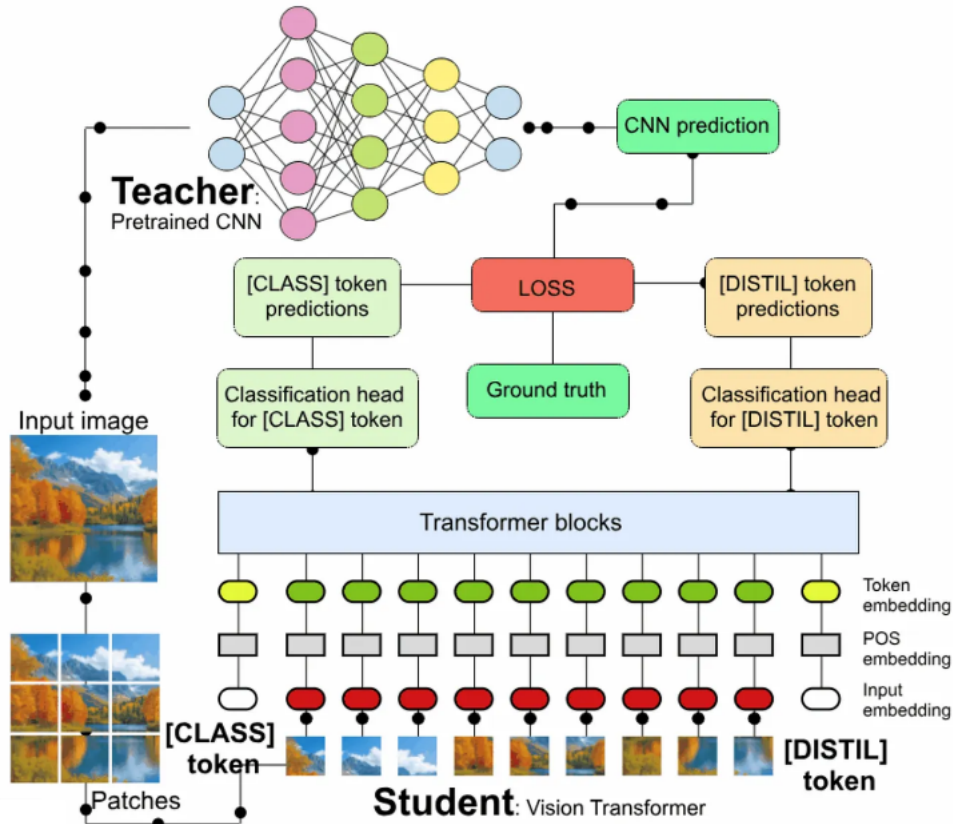


Figure 8.3: DeiT architecture.

Note

In the original DeiT experiments, the default teacher was RegNetY-16GF, trained on ImageNet-1K with the same data and augmentation as the student. Convolutional teachers worked better than Transformer teachers. The authors suggested that transferring some of the CNN's inductive bias could help explain this result, but this is an interpretation rather than a guarantee.

8.3.2 Swin Transformer

The **Swin Transformer (Shifted Window Transformer)** is a landmark architecture introduced by Microsoft Research to solve the two biggest flaws of traditional Vision Transformers (ViTs): explosive computational cost on high-resolution images and a lack of multi-scale feature hierarchies.

When standard Vision Transformers (ViT) treat an image as a sequence of patches, two issues arise:

- **Quadratic complexity $\mathcal{O}(N^2)$:** In standard ViT, every patch attends to every other patch. If you double the image resolution, the number of patches N quadruples, making

attention calculations $16\times$ more expensive. This makes standard ViT impractical for high-resolution vision tasks like segmentation or 4K imaging.

- **Single-scale representations:** ViT keeps feature maps at a uniform resolution throughout all layers. Dense tasks like object detection need fine-grained local details (small scales) alongside broad context (large scales).

The main innovations of the Swin Transformer are:

- **Local window attention (W-MSA):** Instead of letting every token attend to every other token, Swin divides the token feature map into fixed, non-overlapping windows of $M \times M$ tokens. The original Swin commonly uses $M = 7$.

Let the token feature map have size $h \times w$, with $N = hw$ tokens. Global attention contains

$$(hw)^2 = N^2$$

score entries. Window attention uses hw/M^2 windows, and each window contains an $M^2 \times M^2$ attention matrix. Therefore, the total number of score entries is

$$\frac{hw}{M^2} (M^2)^2 = hwM^2 = NM^2.$$

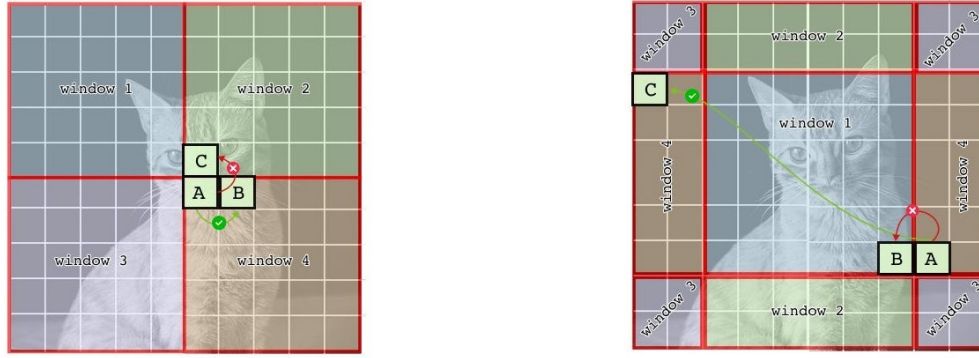
If the image has size $H \times W$ pixels and patch size P , then $h = H/P$ and $w = W/P$, giving

$$\text{global} = \mathcal{O}\left(\frac{H^2W^2}{P^4}\right), \quad \text{windowed} = \mathcal{O}\left(\frac{HWM^2}{P^2}\right).$$

Thus, for a fixed window size M , window attention grows linearly with the number of tokens and global attention quadratically.

- **Shifted window mechanism (SW-MSA):** Calculating self-attention inside isolated windows blocks communication between neighboring regions. Swin connects them by alternating regular and shifted windows in consecutive blocks (see Figure 8.4):
 - Block l (W-MSA): Use the standard window partition.
 - Block $l+1$ (SW-MSA): Shift by $\lfloor M/2 \rfloor$ tokens along both spatial axes so that the new windows cross the previous boundaries.

For efficiency, SW-MSA applies a cyclic shift, computes attention inside the windows, and then reverses the shift. An attention mask prevents false interactions introduced by wrap-around, while a learned relative position bias represents the spatial offset between tokens within each window. SW-MSA has the same asymptotic attention complexity as W-MSA, although shifting and masking add a small overhead.



Regular Window

- A and B are neighbor patches of window 4.
- A and C are neighbor patches of different windows.
- Hence A as a query attends to B as key but A does not attend to C.

Shifted Window

- A and B are geometrically neighbor patches but belong to different windows.
- A and C are geometrically far away patches but they belong to the same window.
- Hence A as a query attends to C as key but A does not attend to B.

Figure 8.4: Regular and shifted windows. The right panel shows the cyclically shifted layout, so apparent distances do not represent the original image geometry. Green checks indicate permitted same-window interactions and red crosses blocked interactions; an attention mask prevents artificial connections caused by wrap-around.

- **Hierarchical feature maps (patch merging):** For an input of shape $h \times w \times D$, Swin groups each 2×2 neighborhood, concatenates its four vectors into a $4D$ vector, and projects it to dimension $2D$. The output has shape $(h/2) \times (w/2) \times 2D$: each spatial axis is halved, the number of tokens is quartered, and the channel dimension is doubled. The original image patches are not resized; instead, each merged token represents a region twice as wide and twice as high. This is why Figure 8.5 shows effective patch sizes growing from $4\times$ to $8\times$ and $16\times$. Patch merging therefore creates the pyramid-style feature hierarchy shown in the figure.

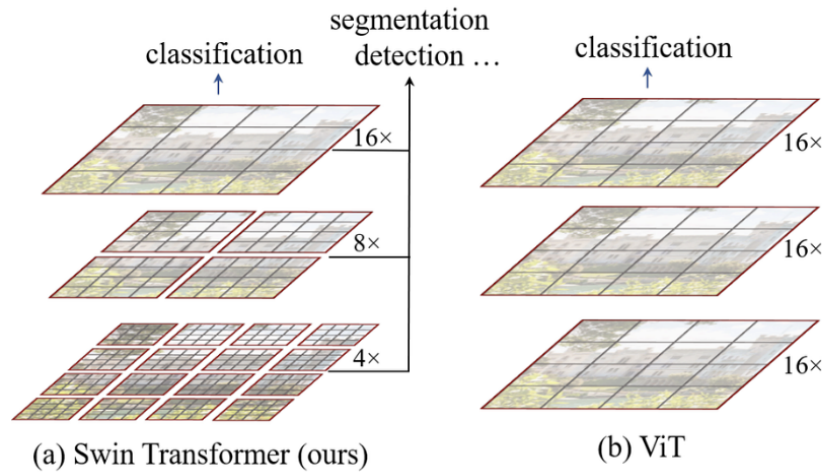


Figure 8.5: Swin Transformer vs Vision Transformer.

8.4 Exercises

Exercise 1: Queries, Keys and Values

Consider independent vectors $q, k \in \mathbb{R}^d$ whose components are mutually independent random variables with mean 0 and variance 1.

1. Prove that the expected value $\mathbb{E}[q \cdot k] = 0$ and the variance $\text{Var}(q \cdot k) = d$.
2. Show that dividing the inner product by $\sqrt{d}$ gives it variance 1, and explain why this helps prevent softmax saturation as d grows.

Exercise 2: Positional Encoding

Explain why self-attention without positional information is permutation-equivariant. Predict how omitting positional embeddings may affect a ViT's ability to distinguish different spatial arrangements of patches.

Exercise 3: CNNs vs ViTs

Compare the main inductive biases of CNNs and ViTs. Discuss how both architecture and training strategy can affect their data efficiency and performance.

Exercise 4: CLS Token

Analyze the role of the [CLS] token in ViT versus Global Average Pooling (GAP) across patch tokens. Explain how the self-attention mechanism enables the [CLS] token to dynamically aggregate global image features.

Exercise 5: Batched Scaled Dot-Product Attention

Write a PyTorch function from scratch that computes multi-head scaled dot-product attention for

$$Q \in \mathbb{R}^{B \times h \times N_q \times d_k}, \quad K \in \mathbb{R}^{B \times h \times N_k \times d_k}, \quad V \in \mathbb{R}^{B \times h \times N_k \times d_v}.$$

Return a tensor of shape (B, h, N_q, d_v) . If an additive mask is provided, use 0 for allowed positions and $-\infty$ for blocked positions. You may compare the result with `F.scaled_dot_product_attention`.

Exercise 6: Patch Embedding Module

Write a PyTorch `nn.Module` that implements image patch tokenization for a fixed image size (H, W) , assuming that P divides both dimensions. Use a 2D convolution to transform (B, C, H, W) into (B, N, D) , and include a learnable [CLS] token and positional embeddings for the resulting $N + 1$ tokens.

Exercise 7: Vision Transformer Encoder Layer

Implement a ViT encoder layer in PyTorch using pre-normalization. Include Multi-Head Self-Attention, LayerNorm, residual connections, and a feed-forward network with GELU and dropout. Preserve the input shape (B, N, D) .

Exercise 8: Distillation Loss

Write a PyTorch loss function for DeiT using either hard distillation with the teacher's predicted class or soft distillation with $\text{KL}(p_t \parallel p_{\text{dist}})$. Combine it with the classification Cross-Entropy loss and treat the teacher output as a fixed target.

Exercise 9: DeiT

Distinguish the DeiT training recipe from its optional distilled variant. Explain the function of the distillation token x_{dist} and why a CNN teacher might be helpful.

Exercise 10: Swin Window Partitioning

Assuming that H and W are divisible by the window size M , write `window_partition` to transform (B, H, W, C) into $(BHW/M^2, M, M, C)$ and `window_reverse` to reconstruct the exact original tensor.

Exercise 11: Swin Transformer

Detail the Shifted Window mechanism (SW-MSA) in the Swin Transformer.